

# Assignment 1

## Instructions

- PHY410: Do problems 1–3
- PHY 505: Do problems 1-4

Accept the assignment from “Classroom 50” website: <https://classroom50.org/ubsuny/compphys-fall26/assignments/assignment-1/accept>. This will create a new repository for you on github, titled something like `github.com/ubsuny/compphys-fall2026-assignment1-username`.

The assignment requires two uploads:

1. **Submit a writeup/lab report to UBLearns.** The writeup should contain your complete solutions to the problems in writing. Feel free to use any format: latex, markdown, etc. For assignments using Jupyter notebooks, you can also do your writeup inside the notebook using Markdown cells, export the notebook to HTML/PDF/etc. (this may require some additional setup), and upload the exported notebook.
2. **Upload your code through Classroom 50.** This just means to `git push` any code you wrote to your GitHub repository. You can push as many times as you like before the due date (in fact, this is a good idea, as it serves as a backup; nothing will be graded before the due date).

## Problem 1

25 points

What are the two's complement representations for the following numbers? Use N=16 bits.

- 10
  - 436
  - 1024
  - -13
  - -1023
  - -1024
- 

## Problem 2

25 points

Suppose I want to compute the following series:

$$f_0 = 2$$
$$f_n = f_{n-1}^2$$

What is the maximum  $n$  that can be stored in the following C++ data types, assuming that an int is 4 bytes, a long int is 8 bytes, and each byte stores 4 bits? Make sure to explain your reasoning in your answer.

- int
  - long int
  - unsigned long int
- 

## Problem 3

25 points

A small C++ program has been provided that demonstrates the series from Problem 2 for the `int` type. The program prints out each element in the series, thus allowing you to see directly what happens when your data type is exceeded.

Importantly, the series is implemented using a simple *recursive function* that ensures that the data type always stays the same. You might be tempted to simplify the program, using the fact that the series is simply  $f_n = 2^{2^n}$  and exploiting the built-in `pow()` function. However, if you check the documentation, you'll see that `pow()` implicitly converts the inputs to floating point numbers, which would ruin your experiment for the desired type!

### Problem 3a

10 points

Compile and execute the program. For example:

```
g++ Problem3.cpp -o Problem3
./Problem3
```

(MacOS users can use `clang++` instead of `g++`; your pixi environment probably installed `clang++` and not `g++`.)

Does the program find the boundary where overflow occurs? If not, modify the program accordingly. In your writeup, include the full output of the program.

### Problem 3b

*15 points*

Does the boundary match what your answer from Problem 2? Explain why or why not.

---

### Problem 4

*25 points*

#### PHY505 students only

PHY410 students are welcome to submit, but it will not count towards the grade.

#### Problem 4a

*15 points*

Copy `Problem3.cpp` to `Problem4.cpp` and modify the new program to test additional data types:

- `int`
- `long int`
- `unsigned long int`

Make sure to extend the  $f_n$  series far enough to find the overflow boundary. In your writeup, include the output of your program and describe the observed differences between the data types. Also, make sure to `git add/commit/push` your new `.cpp` file.

#### Problem 4b

*10 points*

Modify the program (or write a new program) to find the actual maximum value for each data type, on your own computer. You might want to make an educated guess as a starting point (see <https://en.cppreference.com/cpp/language/types>), and focus your search on the known boundaries. Demonstrate the overflow behavior with `std::cout` statements, as in Problem 3.

In your writeup, include the program output, state the maximum value for each type, and determine the number of bits used for each type. Make sure to `git add/commit/push` your C++ code.